GenAi Analysis

Name of AI #1:
Gemini

Accessed:
On my computer by searching for the website.

Prompt:
I am working on EECS 348 Assignment 5. I have attached another student's Assignment 4 C++ Sudoku solver.

Modify the existing code to meet these requirements:

1. Read the five Sudoku puzzle files named puzzle1.txt through puzzle5.txt.
2. Find all possible solutions for each Sudoku puzzle using the existing backtracking approach.
3. Write all solutions for each puzzle to separate files named solution1.txt through solution5.txt.
4. If no solutions are found, write exactly: No solution found.
5. If solutions are found, write all completed Sudoku grids in the same format as the input files, followed by: Solutions found: n
6. Preserve all required functionality from the original Assignment 4.
7. Do not unnecessarily rewrite the entire program. Preserve its existing structure and modify only what is necessary.
8. Add comments explaining the modifications.

Please provide the complete modified C++ source code and explain which sections you changed.\

Generated Code:
```
/*
EECS 348 Assignment 5 - Sudoku Solver (All Solutions to Files)

Description: Reads five Sudoku puzzle files (puzzle1.txt through puzzle5.txt),
finds all possible solutions using recursive depth-first search with backtracking,
and writes the results to separate output files (solution1.txt through solution5.txt).

Input: Sudoku puzzle text files containing numbers and _ for blanks.
Output: Writes all valid solutions or "No solution found" to solution1.txt through solution5.txt.

Collaborators: None
Author: Ismail (Modified for Assignment 5)
*/
```

```
#include <array> // Includes fixed-size container library
#include <fstream> // Includes file stream library for reading/writing files
#include <iostream> // Includes input-output library
#include <string> // Includes string library for text handling

class SudokuPuzzle { // Declares the SudokuPuzzle class for grid management
private: // Begins private member section
   enum { SIZE = 9, BOX_SIZE = 3, CELL_COUNT = SIZE * SIZE }; // Sets board dimension
constants
   using Grid = std::array<std::array<int, SIZE>, SIZE>; // Creates type alias for 9x9 grid

   Grid original_{}; // Initializes grid to store original puzzle values
   Grid board_{}; // Initializes working board grid for search
   int solutionCount_ = 0; // Modified for Assignment 5: Tracks total number of solutions found

   void printGrid(const Grid& grid, std::ostream& out) const {
      for (int row = 0; row < SIZE; ++row) { // Loops through each row of the grid
         for (int col = 0; col < SIZE; ++col) { // Loops through each column of the grid
            if (col != 0) { // Checks if not the first column
               out << ' '; // Prints space between grid cells
            }
            if (grid[row][col] == 0) { // Checks if the cell is empty
               out << '_'; // Prints underscore for empty cell
            } else {
               out << grid[row][col]; // Prints the numeric digit
            }
         }
         out << '\n'; // Prints a newline at the end of the row
      }
   }

   bool canPlace(int row, int col, int value) const {
      for (int index = 0; index < SIZE; ++index) { // Loops through row and column indices
         if (index != col && board_[row][index] == value) { // Checks row for duplicate value
            return false; // Returns false if value already exists in row
         }
         if (index != row && board_[index][col] == value) { // Checks column for duplicate value
            return false; // Returns false if value already exists in column
         }
      }

      const int firstRow = (row / BOX_SIZE) * BOX_SIZE; // Calculates starting row of 3x3 block
      const int firstCol = (col / BOX_SIZE) * BOX_SIZE; // Calculates starting column of 3x3
block
```

```
        for (int r = firstRow; r < firstRow + BOX_SIZE; ++r) { // Loops through 3x3 block rows
            for (int c = firstCol; c < firstCol + BOX_SIZE; ++c) { // Loops through 3x3 block columns
                if ((r != row || c != col) && board_[r][c] == value) { // Checks 3x3 block for duplicate
value
                    return false; // Returns false if value exists in block
                }
            }
        }
        return true; // Returns true if placement is valid
    }

    void search(std::ostream& out) {
        int chosenRow = -1; // Initializes chosen row variable
        int chosenCol = -1; // Initializes chosen column variable
        int fewestCandidates = SIZE + 1; // Initializes minimum candidate tracker
        std::array<int, SIZE> chosenCandidates{}; // Initializes array for best cell candidates

        for (int row = 0; row < SIZE; ++row) { // Loops through all rows
            for (int col = 0; col < SIZE; ++col) { // Loops through all columns
                if (board_[row][col] != 0) { // Checks if cell is already filled
                    continue; // Skips non-empty cells
                }
                std::array<int, SIZE> candidates{}; // Creates candidate container
                int count = 0; // Initializes candidate count
                for (int value = 1; value <= SIZE; ++value) { // Loops through numbers 1 to 9
                    if (canPlace(row, col, value)) { // Checks if number can be placed
                        candidates[count++] = value; // Stores valid candidate digit
                    }
                }
                if (count == 0) { // Checks if cell has zero valid candidates
                    return; // Returns early to trigger backtracking
                }
                if (count < fewestCandidates) { // Checks if current cell has fewer candidates
                    chosenRow = row; // Updates selected row
                    chosenCol = col; // Updates selected column
                    fewestCandidates = count; // Updates minimum candidate count
                    chosenCandidates = candidates; // Updates saved candidate list
                }
            }
        }

        if (chosenRow == -1) { // Checks if all cells are filled
            solutionCount_++; // Modified for Assignment 5: Increments solution counter
            printGrid(board_, out); // Prints completed board in input format
```

```
            out << '\n'; // Prints newline separating solutions
            return; // Returns from current recursive branch to continue searching
        }

        for (int index = 0; index < fewestCandidates; ++index) { // Loops through valid candidates
            board_[chosenRow][chosenCol] = chosenCandidates[index]; // Places candidate on
board
            search(out); // Recurses to solve the next cell
            board_[chosenRow][chosenCol] = 0; // Resets cell back to empty during backtrack
        }
    }

public:
    bool load(const std::string& filename, std::string& error) {
        error.clear(); // Clears any previous error message
        original_ = Grid{}; // Resets original puzzle grid
        board_ = Grid{}; // Resets working puzzle grid
        solutionCount_ = 0; // Resets solution counter
        std::ifstream input(filename); // Opens file stream for reading
        if (!input) { // Checks if file stream failed to open
            error = "Cannot open puzzle file."; // Sets file open error message
            return false; // Returns false if file cannot be opened
        }

        Grid parsed{}; // Creates temporary grid container
        int count = 0; // Initializes cell counter
        char token = '\0'; // Initializes token character variable
        while (input >> token) { // Reads tokens from file skipping whitespace
            if (count >= CELL_COUNT) { // Checks if file has extra data beyond 81 cells
                error = "Expected exactly 81 cells; found extra data."; // Sets size error message
                return false; // Returns false for malformed input
            }
            if (token != '_' && (token < '1' || token > '9')) { // Checks for invalid character tokens
                error = "Invalid character found."; // Sets invalid character error message
                return false; // Returns false for invalid input
            }
            parsed[count / SIZE][count % SIZE] = (token == '_') ? 0 : token - '0'; // Maps token to grid
cell
            ++count; // Increments total cell count
        }
        if (input.bad() || (input.fail() && !input.eof())) { // Checks for file stream I/O errors
            error = "An I/O error occurred."; // Sets I/O error message
            return false; // Returns false on stream failure
        }
```

```
        if (count != CELL_COUNT) { // Checks if file has fewer than 81 cells
            error = "Incomplete data found."; // Sets incomplete data error message
            return false; // Returns false for incomplete input
        }
        original_ = parsed; // Stores parsed grid as original givens
        board_ = parsed; // Initializes working board with givens
        return true; // Returns true upon successful loading
    }

    bool validate(std::string& error) const {
        error.clear(); // Clears error string
        for (int row = 0; row < SIZE; ++row) { // Loops through each board row
            for (int col = 0; col < SIZE; ++col) { // Loops through each board column
                const int value = board_[row][col]; // Retrieves cell value
                if (value != 0 && !canPlace(row, col, value)) { // Checks if initial given violates rules
                    error = "Conflicting given found."; // Sets conflict error message
                    return false; // Returns false if initial conflict exists
                }
            }
        }
        return true; // Returns true if all initial givens are valid
    }

    bool printOriginal(std::ostream& out) const {
        printGrid(original_, out); // Calls printGrid helper for original board
        return true; // Returns true
    }

    void solveAll(std::ostream& out) {
        board_ = original_; // Resets working board to original givens
        solutionCount_ = 0; // Resets solution counter
        std::string error; // Creates local error string
        if (!validate(error)) { // Validates starting board state
            out << "No solution found\n"; // Modified for Assignment 5: Writes exact required failure
message
            return; // Returns early if validation fails
        }
        search(out); // Begins recursive backtracking search
        if (solutionCount_ == 0) { // Checks if no solution was found during search
            out << "No solution found\n"; // Modified for Assignment 5: Writes exact required failure
message
        } else {
            out << "Solutions found: " << solutionCount_ << '\n'; // Modified for Assignment 5: Writes
total solutions count
```

```
        }
    }
};

class SudokuApplication {
public:
    int run() const {
        bool inputError = false; // Initializes application error tracker flag

        // Modified for Assignment 5: Loop through puzzle1.txt to puzzle5.txt
        // and write corresponding outputs to solution1.txt to solution5.txt.
        for (int i = 1; i <= 5; ++i) {
            std::string puzzleFilename = "puzzle" + std::to_string(i) + ".txt";
            std::string solutionFilename = "solution" + std::to_string(i) + ".txt";

            SudokuPuzzle puzzle; // Creates SudokuPuzzle object instance
            std::string error; // Initializes error string

            std::ofstream outFile(solutionFilename); // Opens output file stream for solution
            if (!outFile) {
                std::cerr << "Cannot open output file: " << solutionFilename << '\n';
                inputError = true;
                continue;
            }

            if (!puzzle.load(puzzleFilename, error)) { // Attempts to load puzzle file
                outFile << "No solution found\n"; // Writes required failure message on load error
                inputError = true;
                outFile.close();
                continue;
            }

            if (!puzzle.validate(error)) { // Validates starting puzzle givens
                outFile << "No solution found\n"; // Writes required failure message on validation error
                inputError = true;
                outFile.close();
                continue;
            }

            puzzle.solveAll(outFile); // Solves puzzle and writes all valid solutions and count to file
            outFile.close(); // Closes output file stream
        }
        return inputError ? 1 : 0; // Returns exit status code based on errors
    }
```

```
};

int main() {
    SudokuApplication application; // Creates SudokuApplication object instance
    return application.run(); // Runs application and returns exit status code
}
```

Analysis:
The modified Sudoku solver is accurate because it checks the rules and uses a depth-first search with backtracking to find and record all valid solutions. For execution time, the program has a worst case time complexity of O(9^n), but it's more efficient because it eliminates invalid numbers and chooses cells with a strategy. The space complexity is O(1) outside of the 9x9 grid. The program is easy to maintain because the code is organized into the SudokuPuzzle and SudokuApplication classes, uses descriptive method names, and includes comments explaining important parts of the code, such as file input/output and counter updates.

Name of AI #2:
Copilot

Accessed:
On my computer by searching for the website.

Prompt:
I am working on EECS 348 Assignment 5. I have attached another student's Assignment 4 C++ Sudoku solver.

Modify the existing code to meet these requirements:

1. Read the five Sudoku puzzle files named puzzle1.txt through puzzle5.txt.
2. Find all possible solutions for each Sudoku puzzle using the existing backtracking approach.
3. Write all solutions for each puzzle to separate files named solution1.txt through solution5.txt.
4. If no solutions are found, write exactly: No solution found.
5. If solutions are found, write all completed Sudoku grids in the same format as the input files, followed by: Solutions found: n
6. Preserve all required functionality from the original Assignment 4.
7. Do not unnecessarily rewrite the entire program. Preserve its existing structure and modify only what is necessary.
8. Add comments explaining the modifications.

Please provide the complete modified C++ source code and explain which sections you changed.\

Generated Code:

```
/*
EECS 348 Assignment 5 - Sudoku Solver
MODIFICATIONS FOR ASSIGNMENT 5:
1. Reads puzzle1.txt through puzzle5.txt.
2. Finds ALL solutions using the existing backtracking algorithm.
3. Writes solutions to solution1.txt through solution5.txt.
4. Writes exactly "No solution found." when no solution exists.
5. Writes all solution grids followed by:
        Solutions found: n
6. Preserves Assignment 4 functionality and structure.
*/
#include <array>
#include <fstream>
#include <iostream>
#include <string>
class SudokuPuzzle {
private:
        enum { SIZE = 9, BOX_SIZE = 3, CELL_COUNT = SIZE * SIZE };
        using Grid = std::array<std::array<int, SIZE>, SIZE>;
        Grid original_{};
        Grid board_{};
        bool foundSolution_ = false;
        // Assignment 5:
        // Track total number of solutions found.
        int solutionCount_ = 0;
        void printGrid(const Grid& grid, std::ostream& out) const {
        for (int row = 0; row < SIZE; ++row) {
        for (int col = 0; col < SIZE; ++col) {
                if (col != 0) {
                out << ' ';
                }
                if (grid[row][col] == 0) {
                out << '_';
                }
                else {
                out << grid[row][col];
                }
        }
        out << '\n';
        }
        }
        bool canPlace(int row, int col, int value) const {
        for (int index = 0; index < SIZE; ++index) {
```

```
if (index != col && board_[row][index] == value) {
        return false;
}
if (index != row && board_[index][col] == value) {
        return false;
}
}
const int firstRow = (row / BOX_SIZE) * BOX_SIZE;
const int firstCol = (col / BOX_SIZE) * BOX_SIZE;
for (int r = firstRow; r < firstRow + BOX_SIZE; ++r) {
for (int c = firstCol; c < firstCol + BOX_SIZE; ++c) {
        if ((r != row || c != col) &&
        board_[r][c] == value) {
        return false;
        }
}
}
return true;
}
// MODIFIED:
// Still performs recursive backtracking.
// Now stores every solution and counts them.
void search(std::ostream& out) {
int chosenRow = -1;
int chosenCol = -1;
int fewestCandidates = SIZE + 1;
std::array<int, SIZE> chosenCandidates{};
for (int row = 0; row < SIZE; ++row) {
for (int col = 0; col < SIZE; ++col) {
        if (board_[row][col] != 0) {
        continue;
        }
        std::array<int, SIZE> candidates{};
        int count = 0;
        for (int value = 1; value <= SIZE; ++value) {
        if (canPlace(row, col, value)) {
        candidates[count++] = value;
        }
        }
        if (count == 0) {
        return;
        }
        if (count < fewestCandidates) {
        chosenRow = row;
```

```
                chosenCol = col;
                fewestCandidates = count;
                chosenCandidates = candidates;
                }
        }
        }
        // COMPLETE SOLUTION FOUND
        if (chosenRow == -1) {
        foundSolution_ = true;
        ++solutionCount_;
        // Assignment 5:
        // Write solution grid in same format as puzzle input.
        printGrid(board_, out);
        out << '\n';
        return;
        }
        for (int index = 0; index < fewestCandidates; ++index) {
        board_[chosenRow][chosenCol] =
                chosenCandidates[index];
        search(out);
        board_[chosenRow][chosenCol] = 0;
        }
        }
public:
        bool load(const std::string& filename,
        std::string& error) {
        error.clear();
        original_ = Grid{};
        board_ = Grid{};
        foundSolution_ = false;
        solutionCount_ = 0;
        std::ifstream input(filename);
        if (!input) {
        error = "Cannot open puzzle file.";
        return false;
        }
        Grid parsed{};
        int count = 0;
        char token = '\0';
        while (input >> token) {
        if (count >= CELL_COUNT) {
                error =
                "Expected exactly 81 cells; found extra data.";
                return false;
```

```
}
if (token != '_' &&
        (token < '1' || token > '9')) {
        error = "Invalid character found.";
        return false;
}
parsed[count / SIZE][count % SIZE] =
        (token == '_') ? 0 : token - '0';
++count;
}
if (input.bad() ||
(input.fail() && !input.eof())) {
error = "An I/O error occurred.";
return false;
}
if (count != CELL_COUNT) {
error = "Incomplete data found.";
return false;
}
original_ = parsed;
board_ = parsed;
return true;
}
bool validate(std::string& error) const {
error.clear();
for (int row = 0; row < SIZE; ++row) {
for (int col = 0; col < SIZE; ++col) {
        const int value = board_[row][col];
        if (value != 0 &&
        !canPlace(row, col, value)) {
        error = "Conflicting given found.";
        return false;
        }
}
}
return true;
}
bool printOriginal(std::ostream& out) const {
printGrid(original_, out);
return true;
}
// MODIFIED FOR ASSIGNMENT 5
void solveAll(std::ostream& out) {
board_ = original_;
```

```
        foundSolution_ = false;
        solutionCount_ = 0;
        std::string error;
        if (!validate(error)) {
        out << "No solution found.\n";
        return;
        }
        search(out);
        if (!foundSolution_) {
        out << "No solution found.\n";
        }
        else {
        out << "Solutions found: "
                << solutionCount_
                << '\n';
        }
        }
};
class SudokuApplication {
public:
        int run() const {
        const std::array<std::string, 5> filenames = {{
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
        }};
        bool inputError = false;
        for (size_t i = 0; i < filenames.size(); ++i) {
        const std::string& filename = filenames[i];
        std::cout
                << "Puzzle file: "
                << filename
                << '\n';
        SudokuPuzzle puzzle;
        std::string error;
        if (!puzzle.load(filename, error)) {
                std::cout
                << "Error: "
                << error
                << "\n\n";
                inputError = true;
                continue;
```

```
        }
        std::cout << "Original puzzle:\n";
        puzzle.printOriginal(std::cout);
        std::cout << '\n';
        // Assignment 5:
        // Create solution file name.
        std::string solutionFile =
                "solution" +
                std::to_string(i + 1) +
                ".txt";
        std::ofstream output(solutionFile);
        if (!output) {
                std::cout
                << "Error creating "
                << solutionFile
                << '\n';
                inputError = true;
                continue;
        }
        puzzle.solveAll(output);
        std::cout
                << "Results written to "
                << solutionFile
                << "\n\n";
        }
        return inputError ? 1 : 0;
        }
};
int main() {
        SudokuApplication application;
        return application.run();
}
```

Analysis:

The program is good because it keeps the original backtracking algorithm and checks every valid number placement to find and record all completed Sudoku solutions before writing them to the output file. Its execution time is exponential in the worst case, approximately O(9^n), where n is the number of empty cells, because the program may need to test many different number combinations through recursive backtracking. The space complexity is O(n) because the main additional memory usage comes from the recursive call stack. The code is also easy to maintain because it keeps the existing SudokuPuzzle and SudokuApplication classes organized and only makes changes to add solution counting and file output. The added comments and

small changes to the original structure make the new Assignment 5 requirements easier to understand while reducing the chance of breaking the functionality from Assignment 4.

Improvements:

The program could be improved by separating the file-writing code from solveAll, which would make the solver easier to reuse without needing to work with files. Adding exception handling for file operations would also help identify problems when creating or writing to output files. Performance could be improved by saving candidate calculations or making better decisions about which empty cells to solve first. Finally, moving the file loop into its own method or class would make the SudokuApplication class more organized and easier to expand in the future.

Selection choice:

These improvements should be fairly easy to implement because the existing code is already organized using object-oriented programming. The SudokuPuzzle class handles the Sudoku grid and solving logic, while SudokuApplication handles the files and overall program execution. Because these responsibilities are already separated, most changes could be made to specific methods without changing the original backtracking algorithm or the 9x9 grid.